Aim:

To write a Java program to find the **day number of the year** for a given date (i.e., determine which day of the year a given date represents).

Algorithm:

1. Start.
2. Read the **day, month, and year** from the user.
3. Store the number of days in each month in an array.
4. Check if the given year is a leap year.
5. If it is a leap year, set February's days to 29.
6. Add the days of all the months before the given month.
7. Add the given day to the total.
8. Display the day number of the year.
9. Stop.

Program :

```
import java.util.Scanner;


public class DayOfYear {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter day: ");
      int day = sc.nextInt();


      System.out.print("Enter month: ");
      int month = sc.nextInt();
```

```
        System.out.print("Enter year: ");
        int year = sc.nextInt();

        int[] daysInMonth = {31,28,31,30,31,30,31,31,30,31,30,31};

        if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {
            daysInMonth[1] = 29;
        }

        int dayOfYear = day;

        for (int i = 0; i < month - 1; i++) {
            dayOfYear += daysInMonth[i];
        }

        System.out.println("Day of the Year: " + dayOfYear);

        sc.close();
    }
}
```

Output:

Enter day: 15

Enter month: 3

Enter year: 2024

Day of the Year: 75

Aim:

To write a Java program to determine the **day of the week** for a given date.

Algorithm:

1. Start.
2. Read the **day, month, and year** from the user.
3. Create a LocalDate object using the given date.
4. Use the getDayOfWeek() method to find the day of the week.
5. Display the day of the week.
6. Stop.

Program:

```
import java.util.Scanner;
import java.time.LocalDate;
import java.time.DayOfWeek;

public class DayOfWeekProgram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter day: ");
        int day = sc.nextInt();

        System.out.print("Enter month: ");
        int month = sc.nextInt();

        System.out.print("Enter year: ");
        int year = sc.nextInt();
```

```
        LocalDate date = LocalDate.of(year, month, day);
        DayOfWeek dayOfWeek = date.getDayOfWeek();

        System.out.println("Day of the Week: " + dayOfWeek);

        sc.close();
    }
}
```

Output:

Enter day: 5

Enter month: 8

Enter year: 2026

Day of the Week: WEDNESDAY

Aim:

To write a Java program to demonstrate the implementation of a **Priority Queue** using the PriorityQueue class.

Algorithm:

1. Start.
2. Import the PriorityQueue class.
3. Create a PriorityQueue of integers.
4. Insert elements into the priority queue using the add() method.
5. Display the elements of the priority queue.
6. Remove the highest-priority element using the poll() method.
7. Display the updated priority queue.
8. Display the highest-priority element using the peek() method.
9. Stop.

Program:

```
import java.util.PriorityQueue;


public class JavaPriorityQueue {
   public static void main(String[] args) {


      PriorityQueue<Integer> pq = new PriorityQueue<>();


      pq.add(40);
      pq.add(10);
      pq.add(30);
      pq.add(20);
      pq.add(50);
```

```
        System.out.println("Priority Queue: " + pq);

        System.out.println("Removed Element: " + pq.poll());

        System.out.println("Priority Queue after removal: " + pq);

        System.out.println("Top Element: " + pq.peek());
    }
}
```

Output:

Priority Queue: [10, 20, 30, 40, 50]

Removed Element: 10

Priority Queue after removal: [20, 40, 30, 50]

Top Element: 20

Aim:

To write a Java program to demonstrate the implementation of an **ArrayList** using the ArrayList class.

Algorithm:

1. Start.
2. Import the ArrayList class.
3. Create an ArrayList of strings.
4. Add elements to the ArrayList using the add() method.
5. Display the elements of the ArrayList.
6. Remove an element using the remove() method.
7. Display the updated ArrayList.
8. Access an element using the get() method.
9. Stop.

Program:

```
import java.util.ArrayList;


public class JavaArrayList {
   public static void main(String[] args) {


      ArrayList<String> list = new ArrayList<>();


      list.add("Apple");
      list.add("Banana");
      list.add("Orange");
      list.add("Mango");
```

```
        System.out.println("ArrayList: " + list);

        list.remove("Banana");

        System.out.println("ArrayList after removal: " + list);

        System.out.println("Element at index 1: " + list.get(1));
    }
}
```

Output:

ArrayList: [Apple, Banana, Orange, Mango]

ArrayList after removal: [Apple, Orange, Mango]

Element at index 1: Orange

Aim:

To write a Java program to find the **largest of three numbers**.

Algorithm:

1. Start.
2. Read three numbers from the user.
3. Compare the three numbers using if-else statements.
4. Store the largest number.
5. Display the largest number.
6. Stop.

Program:

```
import java.util.Scanner;


public class LargestNumber {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter first number: ");
      int a = sc.nextInt();


      System.out.print("Enter second number: ");
      int b = sc.nextInt();


      System.out.print("Enter third number: ");
      int c = sc.nextInt();


      int largest;
```

```
        if (a >= b && a >= c) {
            largest = a;
        } else if (b >= a && b >= c) {
            largest = b;
        } else {
            largest = c;
        }

        System.out.println("Largest Number: " + largest);

        sc.close();
    }
}
```

Output:

Enter first number: 25

Enter second number: 48

Enter third number: 31

Largest Number: 48

Aim:

To write a Java program to demonstrate the use of the **Comparator interface** for sorting objects based on a specific property.

Algorithm:

1. Start.
2. Import the required Java utility classes.
3. Create a Student class with name and age attributes.
4. Create a Comparator class to compare students based on their age.
5. Create an ArrayList of Student objects.
6. Add student objects to the list.
7. Sort the list using the comparator with the sort() method.
8. Display the students in ascending order of age.
9. Stop.

Program:

```
import java.util.*;


class Student {
    String name;
    int age;


    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}


class AgeComparator implements Comparator<Student> {
```

```
    public int compare(Student s1, Student s2) {
        return Integer.compare(s1.age, s2.age);
    }
}

public class JavaComparator {
    public static void main(String[] args) {

        ArrayList<Student> list = new ArrayList<>();

        list.add(new Student("Ravi", 22));
        list.add(new Student("Arun", 19));
        list.add(new Student("Kiran", 21));

        list.sort(new AgeComparator());

        System.out.println("Students sorted by age:");

        for (Student s : list) {
            System.out.println(s.name + " - " + s.age);
        }
    }
}
```

Output:

Students sorted by age:

Arun – 19

Kiran - 21

Ravi - 22

Aim:

To write a Java program to **sort the names of people based on their heights in descending order**.

Algorithm:

1. Start.
2. Read the names and corresponding heights.
3. Compare the heights of the people.
4. Swap both the heights and the corresponding names whenever a taller person is found.
5. Repeat until all names are sorted in descending order of height.
6. Display the sorted names.
7. Stop.

Program:

```
public class SortThePeople {
    public static void main(String[] args) {

        String[] names = {"Mary", "John", "Emma"};
        int[] heights = {180, 165, 170};

        for (int i = 0; i < heights.length - 1; i++) {
            for (int j = i + 1; j < heights.length; j++) {
                if (heights[i] < heights[j]) {
                    int tempHeight = heights[i];
                    heights[i] = heights[j];
                    heights[j] = tempHeight;

                    String tempName = names[i];
                    names[i] = names[j];
```

```
                names[j] = tempName;
            }
        }
    }

    System.out.println("People sorted by height:");

    for (String name : names) {
        System.out.println(name);
    }
  }
}
```

Output:

People sorted by height:

Mary

Emma

John

Aim:

To write a Java program to **filter sensor readings with temperature greater than 50, group them by Sensor ID, compute the average temperature for each sensor, and display the sensors sorted in descending order of average temperature.**

Algorithm:

1. Start.
2. Read the number of sensor readings N.
3. Create a map to store the total temperature and count for each Sensor ID.
4. Read each Sensor ID and temperature.
5. If the temperature is greater than 50:
    - Add the temperature to the sensor's total.
    - Increment the sensor's count.
6. Compute the average temperature for each Sensor ID.
7. Store the results in a list.
8. Sort the list in descending order of average temperature.
9. Display the Sensor ID and its average temperature.
10. Stop.

Program:

```
import java.util.*;


public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      int N = sc.nextInt();


      Map<String, int[]> map = new HashMap<>();
```

```
        for (int i = 0; i < N; i++) {
            String sensorId = sc.next();
            int temp = sc.nextInt();

            if (temp > 50) {
                map.putIfAbsent(sensorId, new int[2]);
                map.get(sensorId)[0] += temp;
                map.get(sensorId)[1]++;
            }
        }

        List<Map.Entry<String, int[]>> list = new ArrayList<>(map.entrySet());

        list.sort((a, b) -> {
            double avgA = (double) a.getValue()[0] / a.getValue()[1];
            double avgB = (double) b.getValue()[0] / b.getValue()[1];
            return Double.compare(avgB, avgA);
        });

        for (Map.Entry<String, int[]> entry : list) {
            double avg = (double) entry.getValue()[0] / entry.getValue()[1];
            System.out.printf("%s %.2f%n", entry.getKey(), avg);
        }

        sc.close();
    }
}
```

Input:

6

S1 45

S2 60

S1 70

S3 80

S2 55

S3 40

Output:

S3 80.00

S1 70.00

S2 57.50